

# Speech-to-Text — in-browser dictation for Re-write Practice

---

How the **Speak** button on the doctor dashboard turns a spoken sentence into text in the Re-write Practice box. All inference runs **inside the browser tab**; no audio is uploaded, stored, or sent to any third party. If this document diverges from the code, the code wins.

Scope: `app/Webapp` only. Nothing in the Backend, the database, or the AI pipeline participates in transcription.

---

## 1. What it does

---

On the doctor dashboard's topic detail view, the Re-write Practice panel asks *"How would you say it better?"*. Next to that prompt sits a microphone button. Clicking it opens the microphone, loads two ONNX models into a Web Worker, and from then on every complete spoken sentence is appended to the textarea as text. Clicking again releases the microphone.

Typing is untouched. Dictation **appends** to whatever is already in the box, so the two can be mixed freely. The recognised text reaches the Backend only where typed text already did — through the unchanged **Try & Score** and `/api/doctor/rewrites` calls.

---

## 2. Design premise — the audio never leaves the tab

---

This is the single decision everything else follows from.

Two conventional options were rejected:

| Option                                                                        | Why not                                                                                         |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Browser <code>SpeechRecognition</code> / <code>webkitSpeechRecognition</code> | Streams the microphone to a browser-vendor service. Absent in Firefox, prefixed in Safari.      |
| A server-side transcription endpoint                                          | Creates an upload path for consultation speech, and therefore a vendor boundary to argue about. |

Instead the model runs client-side via `@huggingface/transformers` (Transformers.js) on ONNX Runtime Web. The consequences:

- **PHI question is moot by construction.** The path is microphone → `AudioContext` → Web Worker → text. There is no upload to disable, so no BAA is required and no vendor boundary exists.
  - **Nothing is stored.** No disk, no IndexedDB, no network. The only cached artefact is the model weights, in the browser's Cache API, on first visit.
  - **The cost moves to the client.** A ~123 MB one-time download and CPU inference on the doctor's own machine. Section 8 records what that actually costs.
- 

## 3. Files

---

Six files, one per responsibility. None exceeds 280 lines.

| File                                 | Lines | Role                                                                                                 |
|--------------------------------------|-------|------------------------------------------------------------------------------------------------------|
| src/components/RewriteVoiceInput.tsx | 108   | The button. Presentation only — label, colours, disabled state, error text.                          |
| src/hooks/useSpeechToText.tsx        | 222   | Browser plumbing: permission, <code>AudioContext</code> , worklet, microphone lifetime, cleanup.     |
| src/lib/sttWorkerHost.ts             | 41    | Owns the worker's lifetime — one per page visit, shared by every dictation.                          |
| src/workers/stt.worker.ts            | 276   | Both models, all inference, and the sentence-boundary state machine.                                 |
| public/vad-processor.js              | 113   | <code>AudioWorklet</code> on the audio rendering thread: re-chunks and rate-converts the mic stream. |
| src/lib/sttConstants.ts              | 89    | Model ids, every tuning constant, the worker message union, <code>appendTranscript()</code> .        |

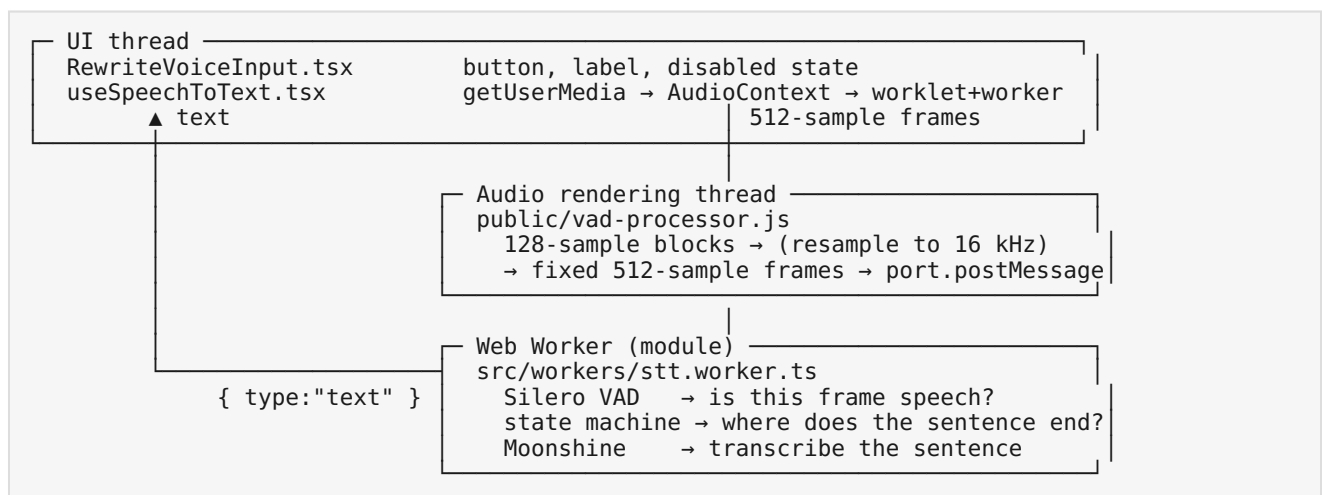
The split between the hook and `sttWorkerHost.ts` is the important one: the **microphone** belongs to a single dictation and is released the moment it ends, while the **models** belong to the visit. Putting both in the hook meant a component unmount — which happens every time a doctor moves between sentences — threw the models away too.

Consumer: `src/components/PhysicianReportsModifiedV41Timothy.tsx` imports the button (:33) and renders it (:4595) in the footer row of the re-write input box, feeding `handleVoiceText` (:3886).

Test coverage: `src/__tests__/lib/sttConstants.test.ts` (unit, `appendTranscript`) and `e2e/voice-input-cross-browser.spec.ts` (Playwright, three engines — see §9).

## 4. Threads and data flow

Three execution contexts, deliberately. Inference on the UI thread would stall the dashboard; audio capture anywhere but the audio rendering thread drops frames.



Step by step, from the click:

1. `start()` takes the shared module `Worker` from `sttWorkerHost.ts` — created here on the first dictation of the visit, reused on every later one — attaches its listeners, and posts `{ type: "reset" }` then `{ type: "load" }`. On a first dictation the models begin downloading; on a later one `ready` comes straight back.

2. `getUserMedia({ channelCount: 1, echoCancellation, autoGainControl, noiseSuppression, sampleRate: 16000 })` prompts for the microphone.
3. An `AudioContext` is built at 16 kHz, the worklet module is added, and the mic source is connected to an `AudioWorkletNode` named `vad-processor`.
4. The worklet emits fixed 512-sample `Float32Array` frames on its port. The hook forwards each one, unmodified, to the worker.
5. The worker scores every frame with Silero VAD and accumulates speech into a rolling buffer.
6. When the state machine decides a sentence has ended, the buffer is handed to Moonshine.
7. The worker posts `{ type: "text", text }`. The hook calls `onText`, which the component's consumer turns into `setNewSentence(prev => appendTranscript(prev, text))`.

The only thing that ever crosses from the worker back to the page is a string.

## Worker message protocol

`SttWorkerMessage` in `sttConstants.ts` is the complete contract:

| Message                                     | Meaning                         | Hook's reaction                                                                   |
|---------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------|
| <code>{ type: "loading", message }</code>   | Download started                | (status is already <code>loading</code> )                                         |
| <code>{ type: "progress", progress }</code> | 0-100 across <b>both</b> models | <code>setProgress</code> → <code>Loading... N%</code>                             |
| <code>{ type: "ready" }</code>              | Warm-up inference done          | <code>setStatus("listening")</code>                                               |
| <code>{ type: "speech", active }</code>     | VAD entered/left a sentence     | <code>setSpeaking</code> → mic icon pulses, label reads <code>Listening...</code> |
| <code>{ type: "text", text }</code>         | One recognised sentence         | <code>onText(text)</code>                                                         |
| <code>{ type: "error", message }</code>     | Load or inference failure       | <code>setStatus("error")</code> , message shown beside the button                 |

Three things go the other way:

| Message                        | Meaning                                                                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>{ type: "load" }</code>  | Load both models. Already loaded — which is the normal case after the first dictation — is answered with <code>ready</code> immediately, so the page never waits. |
| <code>{ type: "reset" }</code> | Forget the previous dictation: the rolling buffer, the pre-speech frames, the VAD's recurrent state. Sent at both the start and the end of every dictation.       |
| <code>{ buffer }</code>        | One 512-sample frame from the worklet. No <code>type</code> field; the worker treats any message without one as audio.                                            |

`reset` also bumps a session counter that a transcription in flight compares against when it finishes. Without it, a sentence whose inference was still running when the doctor pressed Stop would appear in the box a second or two later.

## 5. The two models

### 5.1 Silero VAD — the gate

`onnx-community/silero-vad`, 2.2 MB, loaded at **fp32** unconditionally (quantizing a 2 MB model buys nothing). It scores each 512-sample frame for "is this speech?" and carries a recurrent state tensor of shape `[2, 1, 128]` between frames.

Its purpose is economic: Moonshine is expensive, so it must only ever see audio that is actually someone talking. Without a VAD, the transcriber either runs continuously on silence or has to be driven by a push-to-talk the doctor must remember to release.

### 5.2 Moonshine — the transcriber

`onnx-community/moonshine-base-ONNX`, loaded through the standard `pipeline("automatic-speech-recognition", ...)` API.

**Why Moonshine and not Whisper — it is compute shape, not file size.** Whisper pads every input to a fixed 30-second mel spectrogram, so a five-second dictation costs thirty seconds of encoder work regardless. Moonshine consumes the raw waveform at its natural length (rotary position embeddings, so nothing forces padding), and encoder cost tracks the actual utterance. On a GPU that difference is hidden; on a CPU — which is what a clinician's laptop is — it is the difference between usable and not. Short dictated phrases are exactly Moonshine's case.

Measured ONNX download sizes (encoder fp32 + merged decoder q8, i.e. the WASM combination):

| Model                          | encoder      | decoder (merged) | total         |
|--------------------------------|--------------|------------------|---------------|
| <b>moonshine-base</b> (chosen) | fp32 80.8 MB | q8 42.5 MB       | <b>123 MB</b> |
| moonshine-base, int8 encoder   | 20.5 MB      | 42.5 MB          | 63 MB         |
| moonshine-tiny                 | 7.9 MB       | 20.2 MB          | 28 MB         |
| whisper-base                   | 82.5 MB      | 53.7 MB          | 136 MB        |
| whisper-tiny.en                | 32.9 MB      | 30.7 MB          | 64 MB         |
| silero-vad (always loaded)     | —            | 2.2 MB           | 2.2 MB        |

Both families use the identical pipeline call, and the model id is a single constant (`STT_MODEL_ID`), so switching is a one-line change if load time or quality argues for it.

### 5.3 Backend and precision — `DEVICE_DTYPE`

```
const DEVICE_DTYPE = {
  webgpu: { encoder_model: "fp32", decoder_model_merged: "q4" },
  wasm:   { encoder_model: "fp32", decoder_model_merged: "q8" },
} as const;
```

The worker probes `navigator.gpu.requestAdapter()` at load time and picks `webgpu` if an adapter is actually granted, otherwise `onnxruntime-web`'s WebAssembly backend on the CPU. Two separate decisions are encoded here:

- **Encoder stays fp32 on both backends.** It runs *once* per utterance. Quantizing it trades accuracy for a saving on the cheaper half of the work.
- **Decoder quantization depends on the backend.** The decoder runs once per output token, so it dominates. On WebGPU, `q4` wins: memory bandwidth is the bottleneck, and dequantizing 4-bit weights is essentially free inside a shader. On WASM there are no 4-bit kernels — the

runtime would dequantize in scalar code, making `q4` *slower* than `q8`, and on some builds unsupported outright. Hence `q8` there.

In short: **WebGPU runs the decoder at q4, WASM runs it at q8**, and the encoder is full precision either way.

No GPU is required. Every latency figure in §8 was measured on the **CPU/WASM path**.

## 6. Finding sentence boundaries

This is the part that carries the user-visible behaviour, and it lives in `stt.worker.ts`'s `onmessage` handler. The constants are all in `sttConstants.ts`.

| Constant                                 | Value                                         | What it buys                                               |
|------------------------------------------|-----------------------------------------------|------------------------------------------------------------|
| <code>SAMPLE_RATE</code>                 | 16 000 Hz                                     | Rate both models were trained at.                          |
| <code>NEW_BUFFER_SIZE</code>             | 512 samples (32 ms)                           | Frame size Silero VAD expects.                             |
| <code>SPEECH_THRESHOLD</code>            | 0.3                                           | Score above this <b>enters</b> speech.                     |
| <code>EXIT_THRESHOLD</code>              | 0.1                                           | While recording, score above this <b>stays</b> in speech.  |
| <code>MIN_SILENCE_DURATION_MS</code>     | 400 ms                                        | Silence shorter than this is a breath, not a sentence end. |
| <code>MIN_SPEECH_DURATION_SAMPLES</code> | 250 ms                                        | Captures shorter than this are discarded as noise.         |
| <code>SPEECH_PAD_MS</code>               | 80 ms                                         | Padding appended to the captured chunk.                    |
| <code>MAX_NUM_PREV_BUFFERS</code>        | $\text{ceil}(80 \text{ ms} / 512) = 3$ frames | Pre-roll FIFO, see below.                                  |
| <code>MAX_BUFFER_DURATION</code>         | 30 s                                          | Ceiling on one transcription.                              |

**Dual threshold (hysteresis).** Entering speech needs 0.3; staying in it only needs 0.1. A single threshold makes the VAD flap on quiet syllables — the tail of a word dips below the line and the sentence is cut in half. Two thresholds mean it is harder to start than to continue, which is the correct asymmetry for dictation.

**Pre-roll FIFO.** While no speech is detected, the last three frames are kept in `prevBuffers`. When a sentence eventually starts, VAD has already consumed the first ~100 ms of it deciding whether it was speech — so `dispatchSentence()` prepends those kept frames. Without this the transcriber never hears the first syllable, and "prostate" arrives as "rostate".

**End of sentence.** Once recording, non-speech frames accumulate in `postSpeechSamples`. Below `MIN_SILENCE_DURATION_SAMPLES` (400 ms) nothing happens — the speaker is drawing breath. At or above it, the sentence is over.

**Noise rejection.** If the whole capture is shorter than 250 ms it is thrown away without transcribing: a cough, a chair, a door.

**The 30-second ceiling.** If someone talks continuously past the buffer, the buffer is dispatched as-is and the overflow samples are copied to the front of the fresh buffer via `reset(overflow.length)`, so the continuing sentence is not clipped at the seam.

**Serialized inference.** `Transformers.js` cannot run two inferences concurrently, so both VAD and transcription go through a single `inferenceChain` promise. VAD on a 512-sample frame is cheap

enough that queueing behind a transcription does not lose audio — frames keep arriving from the worklet and simply wait.

---

## 7. Browser-specific behaviour

---

### 7.1 Secure-context gate

`getUserMedia` exists only in a secure context. Over plain HTTP on a LAN address, `navigator.mediaDevices` is **undefined** — the API is absent, not merely denied, so no application code can work around it.

The hook probes this on mount and sets `unsupportedReason`, which makes the button render **disabled** with the reason in its tooltip ("Voice input needs a secure connection (HTTPS or localhost).") rather than failing at the click. Typing is untouched in that state.

In this deployment that means voice input works on `https://<host>:3443` (the `webapp-tls` nginx container) and **does not work, by design, on** `http://<host>:3001`.

### 7.2 Firefox — sample-rate fallback and resampling

A 16 kHz `AudioContext` is the first choice, because an engine that accepts it resamples the microphone itself with a better filter than anything affordable on the audio thread. Chromium and WebKit do accept it.

Firefox does not. It ignores the `sampleRate` constraint passed to `getUserMedia` (the track settings do not even report a rate) and then throws on `createMediaStreamSource`:

```
AudioContext.createMediaStreamSource: Connecting AudioNodes from AudioContexts with
different sample-rate is currently not supported.
```

So the hook wraps that one call in a `try/catch`: on failure it closes the context, rebuilds it at the device rate, and lets the worklet convert. Chromium and WebKit never reach the catch, and their path is unchanged.

`vad-processor.js` then does the conversion, with `RESAMPLE_RATIO = sampleRate / TARGET_SAMPLE_RATE` (1 when the context is already 16 kHz, in which case `toTargetRate()` returns its input untouched). Each output sample is the **mean of the input samples it spans**, not a point sample. That average is a crude low-pass filter; plain decimation would fold everything above 8 kHz back into the speech band as aliasing. `resampleCarry` and `resampleOffset` persist across `process()` calls because 128-sample blocks do not divide evenly by the ratio.

### 7.3 Why the worklet is a static file

`audioWorklet.addModule()` takes a plain URL and the worklet scope has no module system, so `vad-processor.js` is served from `public/` rather than bundled. The consequence is that it cannot `import` — `TARGET_SAMPLE_RATE = 16000` is duplicated there and must be kept in sync with `SAMPLE_RATE` in `sttConstants.ts`.

### 7.4 Build configuration

Two obstacles were fixed at the root rather than worked around:

- `.npmrc` sets `onnxruntime-node-install-cuda=skip`. `@huggingface/transformers` pulls in `onnxruntime-node`, whose postinstall downloads native binaries and fails outright on a CUDA 11

host. Inference is browser-side, so that binding is never loaded; `next.config.js` also aliases `onnxruntime-node` and `sharp` to `false`.

- A small webpack plugin in `next.config.js` marks `onnxruntime-web`'s pre-built `ort.bundle.min.*.mjs` as `minimized: true`. Terser otherwise dies on its top-level `import.meta`. The file is genuinely already minified, so this is a statement of fact rather than a suppression.

The transformers/ORT chunks are **not** in the eager set for `/` — `app-build-manifest.json` confirms they load only inside the worker, so First Load JS stays at 387 kB.

## 8. Performance, measured

Measured on the deployed TLS door, 18 sentences over one warm session, `hardwareConcurrency: 52`, `crossOriginIsolated: false`, on the **CPU/WASM path** (the test browser exposes `navigator.gpu` but is granted no adapter):

|                                                     |                                                    |
|-----------------------------------------------------|----------------------------------------------------|
| Model ready after the <b>first</b> click of a visit | 9.0 s (of which the 123 MB download is most)       |
| Model ready after <b>every later</b> click          | <b>0.16 s</b> — the worker still holds them        |
| Per-sentence latency, warm, median                  | <b>1.6 s</b> (re-measured on a later build: 1.1 s) |
| Real-time factor                                    | ≈ 0.18 — a 9-second sentence transcribes in 1.6 s  |

Separating the cold and warm numbers is what shows the steady-state cost is under two seconds, so no optimisation is warranted yet. Two levers remain unspent if a slower machine needs them: the WASM backend is currently **single-threaded** (no `C00P/C0EP` headers are sent, so `crossOriginIsolated` is false), and `STT_MODEL_ID` can drop to a 63 MB or 28 MB build.

### Why the second click is free

Two separate caches, often confused:

1. **The weights on disk.** Transformers.js writes them into the browser's Cache API under `transformers-cache` — 8 entries, **123.46 MB** measured: encoder 77.07 MB, quantized decoder 40.53 MB, `tokenizer.json` 3.59 MB, Silero VAD 2.14 MB, four small configs. This survives reloads and restarts, so the 123 MB download happens once per browser, ever.
2. **The models in memory.** Reading those bytes back, rebuilding two ONNX sessions and running the warm-up inference took **3.3 s** — and used to happen on every click, because `stop()` terminated the worker. The worker now outlives the dictation, so it happens once per visit.

The cost of holding them is memory. Process-tree RSS on the deployment, headless Chromium:

|                                | Total    | Attributable to dictation                |
|--------------------------------|----------|------------------------------------------|
| Dashboard open, never dictated | 568 MB   | —                                        |
| Models loaded, transcribing    | 1 214 MB | <b>+645 MB</b>                           |
| After Stop                     | 1 207 MB | <b>+638 MB</b> — deliberately still held |
| Second Speak                   | 1 192 MB | +624 MB                                  |

123 MB on disk becomes ~640 MB resident because the quantized decoder is expanded to floats for computation and `onnxruntime` allocates its own arenas on top. Before this change, Stop returned most of it (dropping to +247 MB) and charged 3.3 s for the next sentence; that trade was taken the other way round on purpose.

The models are still loaded on the first `start()`, not on mount — most doctors reach this screen without ever dictating, and neither the download nor the 640 MB should be spent on them.

## 9. Cross-browser status

`e2e/voice-input-cross-browser.spec.ts` drives the entire path — capability probe, model download, VAD, transcription, and the disabled-over-plain-http case — in three engines. It runs under its own config (`npm run test:e2e:voice`) and is excluded from the ordinary `e2e` run, because the weights are a ~123 MB download per browser.

The microphone is replaced by a `MediaStream` fed from a decoded WAV, defined on `MediaDevices.prototype`, rather than by `--use-file-for-fake-audio-capture` (Chromium-only). Firefox's own fake device emits a tone, which the VAD correctly declines to call a sentence.

| Engine       | Secure ctx | AudioWorklet | Module worker | WASM SIMD | Context rate | Transcript              |
|--------------|------------|--------------|---------------|-----------|--------------|-------------------------|
| Chromium 143 | ✓          | ✓            | ✓             | ✓         | 16 000 Hz    | ✓                       |
| Firefox 145  | ✓          | ✓            | ✓             | ✓         | 16 000 Hz    | ✓ (after the \$7.2 fix) |
| WebKit 26.0  | ✓          | ✓            | ✓             | ✓         | 16 000 Hz    | ✓                       |

**Caveat.** Playwright's WebKit is the WPE port, not macOS Safari. Same engine and same four APIs, but an iPhone or a Mac still deserves one manual check before this is called proven.

Two test inputs are operator-supplied and never committed: `VOICE_TEST_URL` (it carries a de-identified study token) and `VOICE_TEST_AUDIO` (must be synthetic or public-domain speech, never a real consultation recording).

## 10. UI behaviour

**Currently switched off.** `VOICE_INPUT_ENABLED` in `sttConstants.ts` is `false`, which takes the button off the screen and drops its tour step; the models are never fetched, because they only load on a click that can no longer happen. Everything below describes the feature as it behaves when that value is `true` — the one edit needed to bring it back.

The button's label is derived, not stored:

| Condition                                               | Label                                               |
|---------------------------------------------------------|-----------------------------------------------------|
| <code>!supported</code>                                 | Voice unavailable (disabled, reason in the tooltip) |
| <code>status === "loading"</code>                       | Loading... N%                                       |
| <code>status === "listening" &amp;&amp; speaking</code> | Listening... (mic icon pulses)                      |
| <code>status === "listening" &amp;&amp; silent</code>   | Stop                                                |
| otherwise                                               | Speak                                               |

**Placement.** Inside the input box, in a footer row below the text — the border, background and focus ring belong to a wrapper `div`, and both the textarea and the button sit inside it. It is deliberately *not* positioned over the textarea: a textarea's padding is part of its scroll area, so a floating button would have text scrolling underneath it. Being in the wrapper but outside the textarea makes overlap

structurally impossible; measured at 0 px of overlap with the box scrolled to the top and to the middle. The button is left-aligned so it clears the resize handle in the opposite corner.

**Onboarding.** A "Dictate Your Re-write" step in the detail-view tour (`OnboardingTour.tsx`) is anchored to `data-tour='rewrite-voice-button'`, and is spread into the step list only when `VOICE_INPUT_ENABLED` is true — a step whose target is never rendered stalls the tour rather than skipping it. It states the three things the button cannot say for itself: dictated sentences land in the same box that can still be typed in, the audio is never recorded or uploaded, and a greyed-out "Voice unavailable" means the page was opened over plain http.

**Appending, not replacing.** `appendTranscript(previous, chunk)` joins each recognised sentence to the existing contents with a single space, and attaches a leading `,.!?;` without one. It lives in `sttConstants.ts` rather than the hook because it is the one piece of this feature worth unit-testing on its own.

**Cleanup.** `useEffect(() => stop, [stop])` releases the microphone and closes the `AudioContext` if the doctor navigates away mid-dictation — first, so the browser's recording indicator goes out immediately. The worker is *not* terminated there: it is detached and told to reset, and keeps both models for the rest of the visit. Only a worker that has failed unrecoverably is thrown away, by `disposeSttWorker()`.

**Tracking.** Voice usage reuses the existing `rewrite_input` event with `metadata: { source: "voice" }`. `event_type` is a Postgres enum, so a new value would have needed a migration to record the same thing.

---

## 11. Key files

---

| Path                                                                          | What to look at                                                                                                                                                                                   |
|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app/Webapp/src/lib/sttConstants.ts</code>                               | Model ids, every threshold, <code>SttWorkerMessage</code> , <code>appendTranscript()</code>                                                                                                       |
| <code>app/Webapp/src/workers/stt.worker.ts</code>                             | <code>DEVICE_DTYPE</code> (:72), <code>load()</code> (:77), <code>isSpeechFrame()</code> (:131), <code>resetSession()</code> (:175), <code>dispatchSentence()</code> (:186), state machine (:207) |
| <code>app/Webapp/src/lib/sttWorkerHost.ts</code>                              | <code>getSttWorker()</code> , <code>disposeSttWorker()</code> — why the worker outlives a dictation                                                                                               |
| <code>app/Webapp/src/hooks/useSpeechToText.tsx</code>                         | Secure-context probe (:54), Firefox fallback (:170), worklet wiring (:185)                                                                                                                        |
| <code>app/Webapp/public/vad-processor.js</code>                               | <code>toTargetRate()</code> (:45), 512-sample framing (:90)                                                                                                                                       |
| <code>app/Webapp/src/components/RewriteVoiceInput.tsx</code>                  | Button, label logic, disabled state                                                                                                                                                               |
| <code>app/Webapp/src/components/PhysicianReportsModifiedV41Timothy.tsx</code> | <code>handleVoiceText</code> (:3886), render site (:4595)                                                                                                                                         |
| <code>app/Webapp/e2e/voice-input-cross-browser.spec.ts</code>                 | Capability probe, fake microphone, per-engine assertions                                                                                                                                          |
| <code>app/Webapp/next.config.js</code> , <code>app/Webapp/.npmrc</code>       | onnxruntime aliasing and the pre-minified bundle plugin                                                                                                                                           |

---

## 12. Open items

---

1. **Manual Safari check.** WPE WebKit passes; a real Mac or iPhone has not been tried.
2. **COOP/COEP headers** would make onnxruntime-web multi-threaded. Not sent today, so WASM inference is single-threaded.
3. **A lighter model** (63 MB int8-encoder moonshine-base, or 28 MB moonshine-tiny) is a one-line change if a clinician laptop proves too slow.
4. **The certificate warning on :3443.** Removing it needs `_tls/ca.crt` imported on each client, or a hostname under a controlled domain so a publicly trusted certificate can be issued.
5. **TARGET\_SAMPLE\_RATE is duplicated** in `vad-processor.js` because a worklet cannot import. Any change to `SAMPLE_RATE` must be made in both places.
6. **Nothing unloads the models before the tab closes.** Once a doctor has dictated, ~640 MB stays resident for the rest of the visit, even if they never dictate again. An idle timer in `sttWorkerHost.ts` would cap that, at the price of the 3.3 s rebuild for anyone who comes back after it fires. Not added, because no one has reported memory pressure — the number to watch is a clinician laptop, not this server.
7. **The worker is a singleton with no owner check.** One `RewriteVoiceInput` is ever on screen, so two hooks driving the same worker cannot currently happen. If a second dictation surface is added, `sttWorkerHost.ts` needs to arbitrate rather than hand the same worker to both.